

8-1 Multidimensional Array

A multi-dimensional array can be defined as an array that has more than one dimension. Having more than one dimension means that it can grow in multiple directions. Some popular multidimensional arrays include 2D arrays which grows in two dimensions, and 3D arrays which grows in three dimensions.

Syntax of Multidimensional Arrays

```
data_type array_name[s1][s2]...[sn];
```

where **s1**, **s2**,..., **sn** are the size of each dimension.

Example:

```
// Two dimensional array
```

```
int two_d[2][4];
```

```
// Three dimensional array
```

```
int three_d[2][4][8];
```

Multidimensional Arrays Size

For 1D array, the length of the array is simply its size too. But multidimensional arrays have extra dimensions. So, the size of each dimension is considered separately. The size of multidimensional array is the product of all its dimensions' size. It is similar to calculating area in 2D and volume in 3D.

For example, consider the below array:

```
int arr1[2][4];
```

- The array `int arr[2][4]` can store total $(2 * 4) = 8$ elements (product of its dimensions).

Example (1):

Write program to declare a 2D array with 3 rows and 4 columns. Then print its elements

Sol.

```
#include <iostream>
using namespace std;
int main() {
    // declaring and initializing a 2D array with 3 rows and 4 columns
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    // printing the elements of the 2D array

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Output:

1 2 3 4

5 6 7 8

9 10 11 12

8-2 Types of Multidimensional Arrays

We can have any number of dimensions in an array as per requirement, but the complexity of handling them also increases exponentially. That is why, the most widely used multidimensional arrays are:

- Two-Dimensional Array (2D Array)
- Three-Dimensional Array (3D Array)

2D Array

A **two-dimensional array** in C++ is a collection of elements organized the form of rows and columns. It can be visualized as a table or a grid.

	Col1	Col2	Col3	Col4	...
Row1	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	
Row2	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	
Row3	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]	
Row4	arr[3][0]	arr[3][1]	arr[3][2]	arr[3][3]	
⋮					

Access and Update Elements

Elements of a 2D array have to be accessed using row and column indices inside **[] square brackets**. It is similar to matrix element position, but the only difference is that here indexing starts from 0.

The value of any element can be updated by using = **assignment operator**.

```
arr[i][j];
array_name[i][j] = new_value
```

where, **i** is the index of row, **j** is the index of the column, and **new_value** to updated. The range of indexes should be:

- $0 \leq i \leq (\text{row_size} - 1)$
- $0 \leq j \leq (\text{col_size} - 1)$

Any values other than that leads to the segmentation fault.

```
#include <iostream>
using namespace std;
int main() {
    int arr[2][4] = {0, 1, 2, 3, 4, 5, 6, 7};

    // Accessing 3rd element is 1st row
    cout << arr[0][2] << endl;

    // Accessing first element in 2nd row
    cout << arr[1][0] << endl;

    // Updating 3rd element is 1st row
    arr[0][2] = 22;
    cout << arr[0][2] << endl;

    // Updating first element in 2nd row
    arr[1][0] = 99;
    cout << arr[1][0];
    return 0;
}
```

Output :

2

4

22

99

Traverse 2D Array

Two **loops** nested inside each other are needed to traverse a 2D array. First loop is used to move through the rows of 2D array, while other is used to move through columns in each row to access all the elements of the row.

```
#include <iostream>
using namespace std;

int main() {
    int arr[2][4] = {0, 1, 2, 3, 4, 5, 6, 7};

    // Outer loop to move through rows
    for (int i = 0; i < 2; i++) {
```

Output:

0 1 2 3

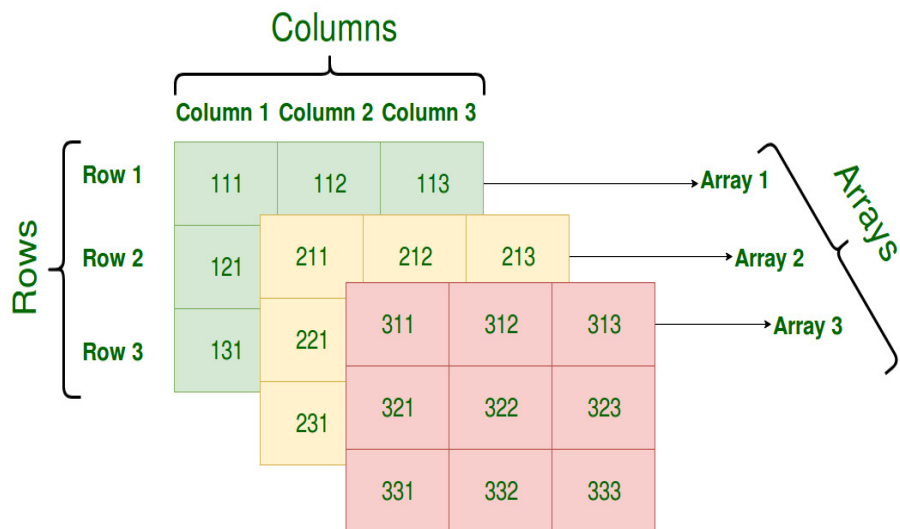
4 5 6 7

```
// Inner loop to move though elements in each row
for (int j = 0; j < 4; j++) {
    cout << arr[i][j] << " ";
}
cout << endl;
}

return 0;
}
```

3D Array

A three-dimensional array in C++ is a collection of elements organized in a 3D cuboid-like structure. It can be visualized as a series of two-dimensional arrays stacked on top of each other.



Create 3D Array

To declare a 3D array in C++, we need to specify its third dimension along with 2D dimensions.

```
int arr[2][2][3];
```

Initialize 3D Array

Like 2D arrays, 3D arrays can also be initialized using a list of values enclosed inside {} curly brackets. However, in a 3D array, the values are grouped into 2D arrays, and each 2D array is nested inside another set of curly brackets.

```
int arr[2][2][3] = {  
    {{0, 1, 2}, {3, 4, 5}},  
    {{6, 7, 8}, {9, 10, 11}}  
};
```

✚ This array has 2 layers (depth), 2 rows per layer, and 3 columns per row. The values are filled sequentially across the layers.

✚ If all the elements are to be initialized to 0, then this syntax can be used:

```
int arr[2][2][3] = {0};
```

This can be only done for 0, not for any other value.

Access and Update Elements

The elements of a 3D array are accessed and updated using three indices: depth, row, and column. These indices must be within the following ranges:

- $0 \leq \text{depth} \leq (\text{depth_size} - 1)$
- $0 \leq \text{row} \leq (\text{row_size} - 1)$
- $0 \leq \text{column} \leq (\text{column_size} - 1)$

```
#include <iostream>  
using namespace std;  
int main() {  
    int arr[2][2][3] = {  
        {{0, 1, 2}, {3, 4, 5}},  
        {{6, 7, 8}, {9, 10, 11}}  
    };  
}
```

```
// Accessing element at depth 0, row 1, column 2  
cout << arr[0][1][2] << endl;
```

Output

```
5  
7  
22  
99
```

```
// Accessing element at depth 1, row 0, column 1
cout << arr[1][0][1] << endl;

// Updating element at depth 0, row 1, column 2
arr[0][1][2] = 22;
cout << arr[0][1][2] << endl;

// Updating element at depth 1, row 0, column 1
arr[1][0][1] = 99;
cout << arr[1][0][1];
return 0;
}
```

Traverse 3D Array

To traverse a 3D array, you need three **nested loops**: one for each dimension (depth, row, column).

Example:

```
#include <iostream>
using namespace std;
int main() {
    int arr[2][2][3] = {
        {{0, 1, 2}, {3, 4, 5}},
        {{6, 7, 8}, {9, 10, 11}}
    };

    // Traverse through all elements of 3D array
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 3; k++) {
                cout << arr[i][j][k] << " ";
            }
            cout << endl;
        }
    }
    return 0;
}
```

Output

0 1 2

3 4 5

6 7 8